

Booleans and Conditional Statements

Prepared by the teaching team for the Fundamentals of Programming course at the University of Science and Technology.

In this lecture, we will cover **Booleans** and **Conditional Statements** in Python, aimed at complete beginners. We will explain how Python uses **boolean values** (`True` and `False`) and how to use `if`, `elif`, `else`, **nested if**, and `match-case` statements to make decisions in code. Each concept is explained step-by-step with simple examples. By the end of this lecture, you will understand how to use boolean logic and conditional statements to control the flow of your Python programs.

Booleans

A *Boolean* is a data type that can only be `True` or `False`, like a light switch that is either ON or OFF. In Python, `True` and `False` are special keywords (case-sensitive). Boolean values represent truth: for example, the question "is 5 greater than 3?" has a True/False answer.

```
a = True
b = False
print(type(a))    # <class 'bool'> - a is of Boolean type
```

Boolean variables are often the result of comparison expressions. For example, comparing numbers or values returns a Boolean:

```
print(10 > 9)    # True (because 10 is greater than 9)
print(5 == 5)    # True (equal to itself)
print(2 <= 1)    # False (2 is not less than or equal to 1)
```

Every comparison operator (`==`, `!=`, `<`, `>`, `<=`, `>=`) yields a Boolean **True** or **False** ¹. These Booleans can be used in decisions (we will see how in `if` statements).

Booleans can also come from logical operations:

```
a = True
b = False
print(a and b)   # False (AND is True only if both are True)
print(a or b)    # True  (OR is True if at least one is True)
print(not a)     # False (NOT flips True to False)
```

Finally, Python treats many values as “truthy” or “falsey” automatically. For example, using the `bool()` function:

```
print(bool("Hello")) # True (non-empty string is True)
print(bool(""))      # False (empty string is False)
print(bool(0))        # False (zero is False)
print(bool(42))       # True (non-zero number is True)
print(bool([]))       # False (empty list is False)
print(bool([1,2]))    # True (non-empty list is True)
```

Most non-empty or non-zero values are **True** by default, and things like `0`, empty sequences (`""`, `[]`, `{}`), or `None` are **False** when converted to Boolean ². Booleans are essential for controlling program flow: conditions in `if` statements evaluate to these truth values ³ ².

Conditional Statements

A *conditional statement* lets the program make decisions: it runs a block of code **only if** a condition is true. The most basic conditional in Python is the `if` statement. The `if` keyword is followed by a condition (a Boolean expression), a colon, and then an indented block of code. That block runs only when the condition is true ⁴.

```
age = 18
if age >= 18:
    print("You are an adult.")
```

Here, the condition `age >= 18` is checked. Since `age` is 18, the condition is **True** and the message is printed. If `age` had been 17, the condition would be False and nothing would print. This is like a fork in the road: *if* the road condition (e.g. rain) is met, take one path; otherwise, skip it.

Indentation matters: Python uses indentation (spaces) to define the block under an `if`. All lines under the `if` must be indented by the same amount (usually 4 spaces) ⁵. For example, the above code must have `print(...)` indented inside the `if`. If you forget the indent, Python will give an error.

Conditional statements are very useful in programs. For example, you might check:

- If a password is correct, allow login; else show an error.
- If a list is empty, load default values; else process the list.
- If a sensor value is above a threshold, trigger an alert.

These are all cases where you want to **do something only when a condition is true**.

<!-- Example of a simple conditional check -->

```
number = -5
if number > 0:
    print("Positive number")
```

In this example, nothing is printed, because `number > 0` is False.

if - elif - else

Often you need more than one condition. Python lets you chain conditions using `elif` (short for “else if”) and `else`. The syntax is:

```
if condition1:
    # block 1
elif condition2:
    # block 2
elif condition3:
    # block 3
else:
    # block 4 (default)
```

- The first `if` is checked; if it is **True**, its block runs and the rest are skipped.
- If it is **False**, Python checks the first `elif` condition. If that is True, its block runs.
- You can have any number of `elif` clauses.
- An optional `else` at the end catches all remaining cases when none of the above conditions were true.

For example, grading logic:

```
score = 85
if score >= 90:
    print("Grade A")
elif score >= 80:
    print("Grade B")
elif score >= 70:
    print("Grade C")
else:
    print("Grade D or below")
```

In this code, Python checks each condition in order: first `score >= 90`, then `score >= 80`, etc. Since `score` is 85, the first condition is False, the second (`score >= 80`) is True, so it prints “Grade B” and skips the rest. The `elif` keyword tells Python “if the previous conditions were not true, then try this one” ⁶. If no `if` or `elif` conditions match, the `else` block (last one) runs.

Only **one** branch runs in an `if/elif/else` chain. It's like a ladder of questions: Python goes down the list and takes the first "yes" answer. The `else` case is a catch-all for any situation not covered by the earlier conditions ⁷.

Nested if

Python allows *nested* `if` statements: placing one `if` inside another. This is useful when you need to check an extra condition only after the first one is true. For example:

```
age = 20
has_id = True

if age >= 18:
    print("You are an adult.")
    if has_id:
        print("ID check passed - you can enter the club.")
    else:
        print("No ID - you cannot enter.")
else:
    print("You are too young to enter.")
```

Here, the second `if has_id:` is inside the first `if age >= 18:` block. The inner check for `has_id` only runs if the outer condition `age >= 18` is true. In this example, Python first checks `age >= 18` (it is True), prints "You are an adult.", **then** checks `has_id` and prints "ID check passed - you can enter the club.". If `age` were less than 18, the outer `else` would run and the inner checks would be skipped.

Note: Nested `if` statements let you make multi-level decisions (each level deeper is an additional question) ⁸. However, too many levels can make code hard to read. Often, simple logical operators (`and`, `or`) can replace a nested `if`. Use nesting when the inner condition really depends on the outer one. For example, W3Schools says *"Use nested if statements when the inner logic is complex or depends on the outer condition."* ⁸.

Another example of nesting from earlier W3Schools content:

```
x = 41
if x > 10:
    print("Above 10,")
    if x > 20:
        print("and also above 20!")
    else:
        print("but not above 20.")
```

The inner `if x > 20:` only runs because `x > 10` was true ⁸.

match-case statements

Python 3.10 introduced **match-case statements**, which are a more powerful way to branch on values (similar to *switch-case* in some other languages). A `match` statement looks at a value (an expression) once, and then matches it against several `case` patterns. The syntax is:

```
match expression:
    case value1:
        # block for value1
    case value2:
        # block for value2
    case _:
        # default block if no other case matched
```

- The value after `match` is evaluated once.
- Python then checks each `case` in order. If it finds a matching case, it runs that block and skips the rest.
- The `case _:` (underscore) is a wildcard that always matches; put it last to serve as a default, “anything else” case ⁹.

This can make code cleaner when you have many exact values to check. For example, deciding a day of the week:

```
day = 3
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case 4:
        print("Thursday")
    case 5:
        print("Friday")
    case 6:
        print("Saturday")
    case 7:
        print("Sunday")
    case _:
        print("Invalid day number")
```

Here, Python compares `day` against each `case`. Since `day` is 3, it matches the `case 3:` and prints “Wednesday”. If `day` were not 1–7, it would fall through to the `case _:` default and print “Invalid day

number.” This works *instead of* many `elif` checks – the match-case selects one code block to execute ¹⁰
¹¹ .

The `_` (underscore) case is important: it catches any values not matched earlier. As the documentation notes, “The value `_` will always match, so it is important to place it as the last case to make it behave as a default case” ⁹ .

You can also combine multiple matches using a pipe `|` (OR) in one case. For example:

```
day = 4
match day:
    case 1 | 2 | 3 | 4 | 5:
        print("Today is a weekday.")
    case 6 | 7:
        print("Today is the weekend!")
    case _:
        print("Invalid day number.")
```

This prints “Today is a weekday.” for any value 1 through 5, “weekend” for 6 or 7, or a default message otherwise ¹² .

Match-case is useful when you have one value or expression and many fixed cases to handle. It is more readable than a long `if-elif` chain in those situations. For tasks like parsing commands or mapping numbers to messages, it provides clear, concise code.

Overall, **Booleans** let us represent simple True/False conditions, and **conditional statements** (`if`, `elif`, `else`, and `match-case`) let our program decide *what to do* based on those conditions. As we’ve seen, comparisons and logical operations produce Boolean values ¹ ³ , which we use in `if` and `match` statements to control the flow. These tools are fundamental for making programs that can react to different situations – for example, checking user input, handling errors, or branching logic in games and applications.
